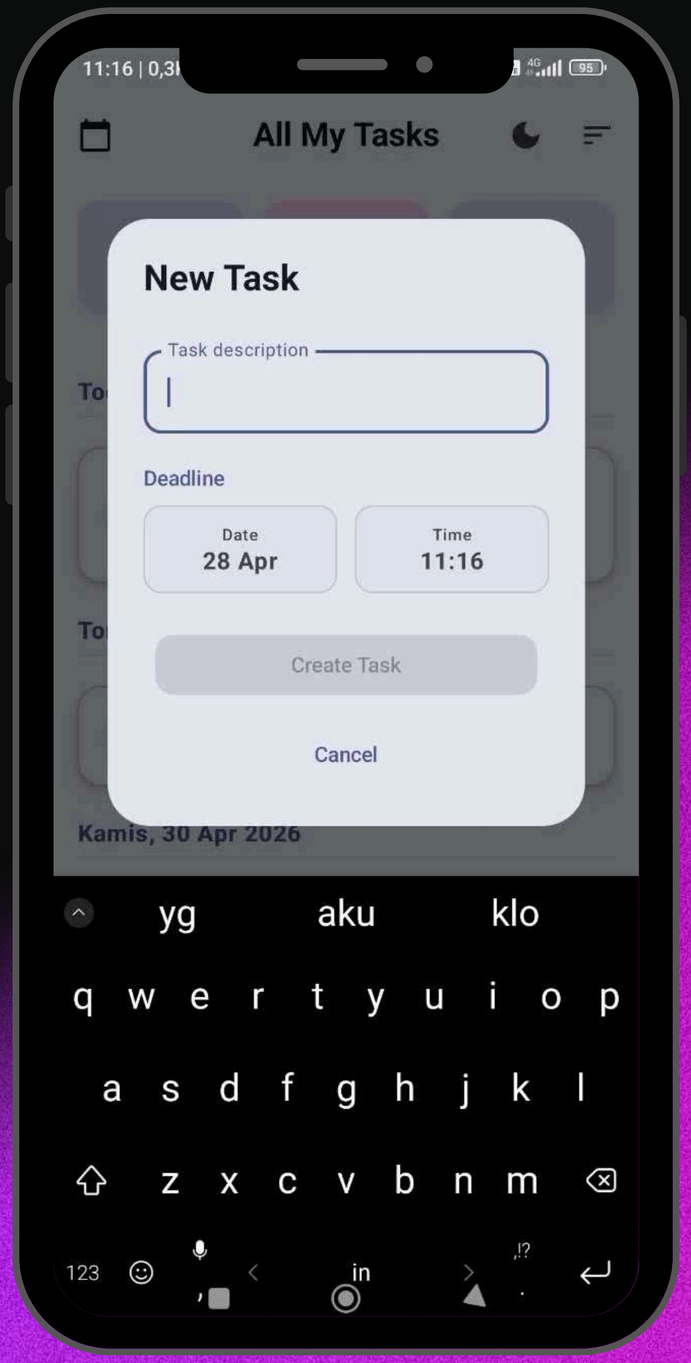
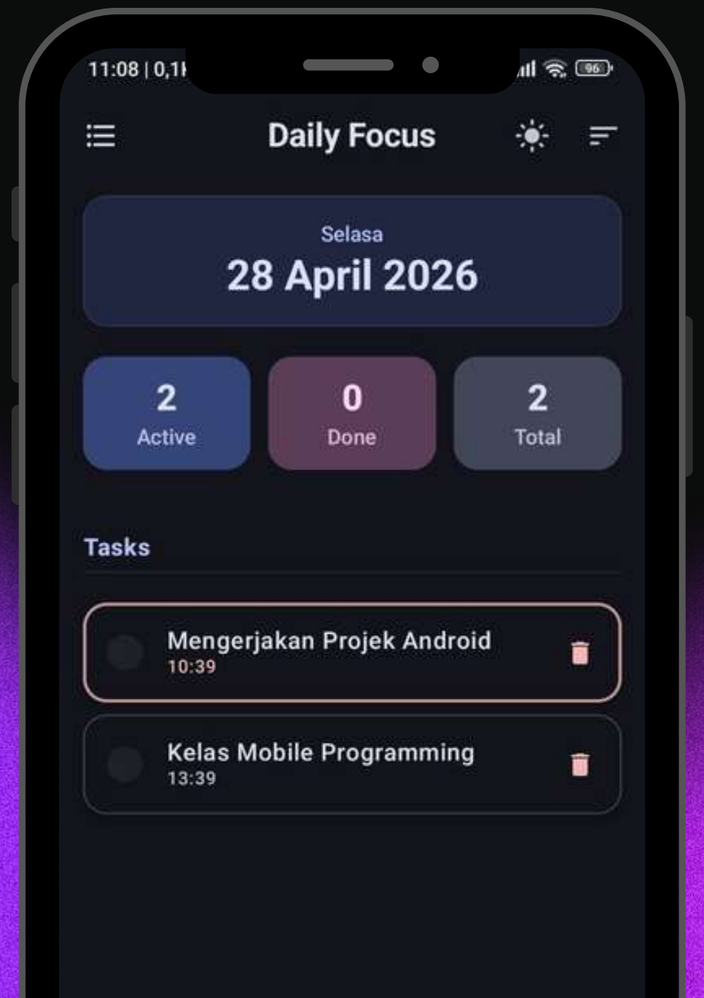
To Do App

Junathan Richie - 5025231019
Faith Mary Sani - 5025231103

Demo Aplikasi



Fitur Aplikasi



Menambahkan, mengedit, menghapus task

Melakukan filter dan sort terhadap task

Terdapat summary task dan *dark mode*

Langkah Pembuatan

Membuat file Task.kt sebagai class data untuk merepresentasikan data setiap Task nantinya

```
package com.example.to_dolistapp

import java.time.LocalDateTime
import java.util.UUID

14 Usages
data class Task(
    val id: String = UUID.randomUUID().toString(),
    val title: String,
    val deadline: LocalDateTime,
    val isCompleted: Boolean = false
)
```

Langkah Pembuatan

Membuat file TaskViewModel.kt sebagai ViewModel menyimpan semua state dan logika bisnis, terpisah dari UI.
Pertama, deklarasi *state-state* dan variabel yang diperlukan dahulu

```
class TaskViewModel : ViewModel() {
    13 Usages
    private val _tasks = mutableStateOf<List<Task>>( value = emptyList())
    1 Usage
    val tasks: State<List<Task>> = _tasks

    3 Usages
    private val _sortOrder = mutableStateOf( value = SortOrder.DEADLINE)
    1 Usage
    val sortOrder: State<SortOrder> = _sortOrder

    2 Usages
    private val _viewMode = mutableStateOf( value = ViewMode.ALL)
    1 Usage
    val viewMode: State<ViewMode> = _viewMode
```

```
2 Usages
private val _selectedDate = mutableStateOf( value = LocalDate.now())
1 Usage
val selectedDate: State<LocalDate> = _selectedDate
2 Usages
private val _isDarkMode = mutableStateOf<Boolean?>( value = null) // null artinya ikut sistem
2 Usages
val isDarkMode: State<Boolean?> = _isDarkMode

1 Usage
fun toggleTheme(dark: Boolean) {
    _isDarkMode.value = dark
}
8 Usages
enum class SortOrder {
    4 Usages
    DEADLINE, STATUS
}
```

Langkah Pembuatan

Tambahkan *dummy data* dari task

```
14 Usages
enum class ViewMode {
    5 Usages
    ALL, DAILY
}

init {
    _tasks.value = listOf(
        Task(
            title = "Mengerjakan Projek Android",
            deadline = LocalDateTime.now().plusHours(2),
            isCompleted = false
        ),
        Task(
            title = "Beli Kopi Susu",
            deadline = LocalDateTime.now().plusDays(1),
            isCompleted = false
        ),
        Task(
            title = "Olahraga ",
            deadline = LocalDateTime.now().minusDays(1),
            isCompleted = true
        ),
        Task(
            title = "Kelas Mobile Programming",
            deadline = LocalDateTime.now().plusHours(5),
            isCompleted = false
        ),
        Task(
            title = "Belajar Jetpack Compose",
            deadline = LocalDateTime.now().plusDays(2),
            isCompleted = false
        )
    )
    sortTasks()
}
```

Langkah Pembuatan

Buat fungsi update task yang berhubungan, seperti

- addTask
- updateTask
- removeTask
- toggleTaskCompletion
- setSortOrder
- setViewMode (dark/light mode)
- setSelectedData
- sortTasks

```
fun addTask(title: String, deadline: LocalDateTime) {
    val newTask = Task(title = title, deadline = deadline)
    _tasks.value = _tasks.value + newTask
    sortTasks()
}

1 Usage
fun updateTask(id: String, title: String, deadline: LocalDateTime) {
    _tasks.value = _tasks.value.map {
        if (it.id == id) it.copy(title = title, deadline = deadline) else it
    }
    sortTasks()
}

1 Usage
fun removeTask(task: Task) {
    _tasks.value = _tasks.value.filter { it.id != task.id }
}

3 Usages
fun toggleTaskCompletion(task: Task) {
    _tasks.value = _tasks.value.map {
        if (it.id == task.id) it.copy(isCompleted = !it.isCompleted) else it
    }
    sortTasks()
}

2 Usages
fun setSortOrder(order: SortOrder) {
    _sortOrder.value = order
    sortTasks()
}

1 Usage
fun setViewMode(mode: ViewMode) {
    _viewMode.value = mode
}
```

Langkah Pembuatan

Membuat UI utama pada MainActivity.kt

```
MainActivity
└── TaskScreen (root)
    ├── TopAppBar
    │   ├── DailyHeader (hanya di mode DAILY)
    │   └── TaskSummaryCard
    │       └── SummaryItem × 3
    ├── LazyColumn
    │   ├── SectionHeader
    │   └── TaskItem × N
    │       └── SwipeToDismissBox
    │           └── Card (isi task)
    ├── TaskDialog (add / edit)
    └── DeleteConfirmDialog
```

Struktur MainActivity.kt

Langkah Pembuatan

Dasar MainActivity.kt, berisi:

- pemanggilan *theme*
- penyimpanan state *dark mode*

```kotlin
class MainActivity : ComponentActivity() {
    2 Usages
    private val viewModel: TaskViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            val isDarkByViewModel by viewModel.isDarkMode
            val darkTheme = isDarkByViewModel ?: isSystemInDarkTheme()
            ToDoListTheme(darkTheme = darkTheme) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    TaskScreen(viewModel)
                }
            }
        }
    }
}
```

Langkah Pembuatan

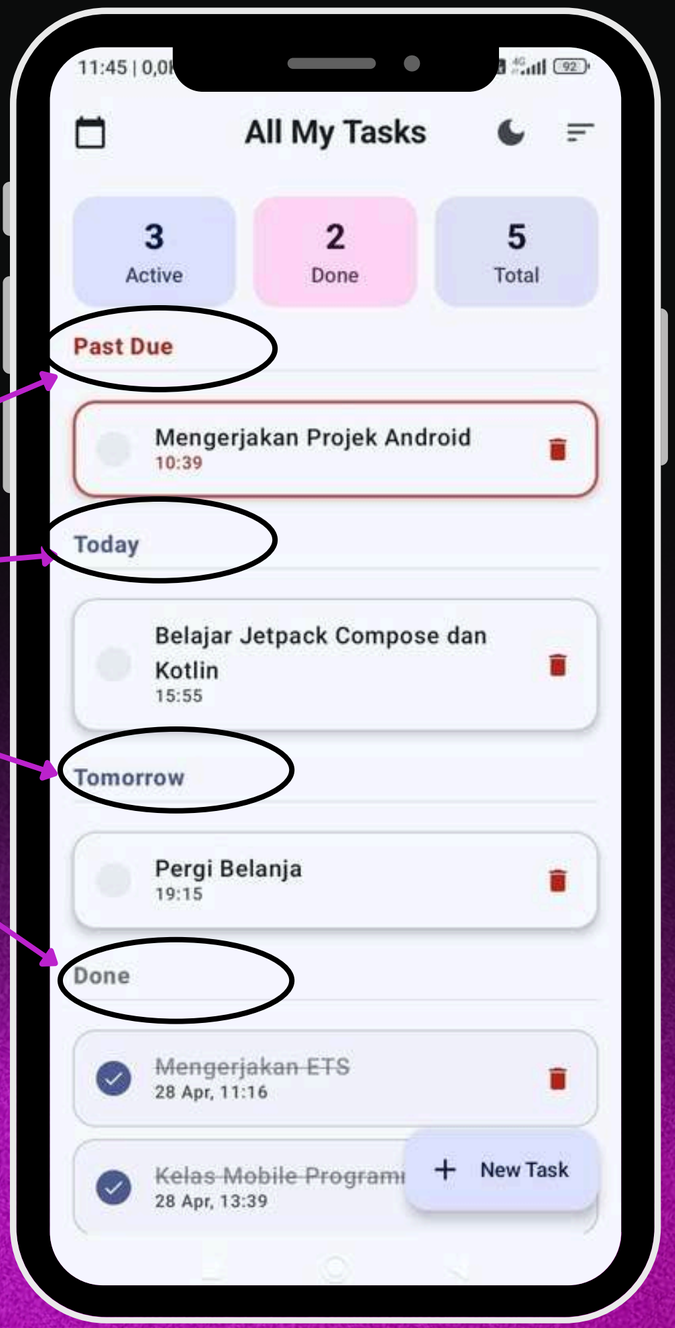
Buat helper *function* untuk menampilkan label tanggal

```
fun getLabelForDate(date: LocalDate): String {
    val today = LocalDate.now()
    return when {
        date == today -> "Today"
        date == today.plusDays( daysToAdd = 1) -> "Tomorrow"
        date.isBefore( other = today) -> "Past Due"
        else -> date.format( formatter = DateTimeFormatter.ofPattern("EEEE, dd MMM yyyy", Locale.getDefault()))
    }
}
```

Langkah Pembuatan

Buat *section header*

```
@Composable
fun SectionHeader(label: String, isError: Boolean = false, isPrimary: Boolean = true) {
    Column(modifier = Modifier.padding(vertical = 12.dp)) {
        Text(
            text = label,
            style = MaterialTheme.typography.titleMedium.copy(
                fontWeight = FontWeight.ExtraBold,
                letterSpacing = 0.5.sp
            ),
            color = when {
                isError -> MaterialTheme.colorScheme.error
                isPrimary -> MaterialTheme.colorScheme.primary
                else -> MaterialTheme.colorScheme.outline
            }
        )
        HorizontalDivider(
            modifier = Modifier.padding(top = 4.dp),
            thickness = 1.dp,
            color = MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f)
        )
    }
}
```



Langkah Pembuatan

Buat summary item dan task summary card
TaskSummaryCard berisi 3 Summary Item (*active*, *done*, dan total)

```
@Composable
fun SummaryItem(
    label: String,
    count: Int,
    modifier: Modifier = Modifier,
    containerColor: androidx.compose.ui.graphics.Color = MaterialTheme.colorScheme.surfaceVariant,
    contentColor: androidx.compose.ui.graphics.Color = MaterialTheme.colorScheme.onSurfaceVariant
) {
    Surface(
        modifier = modifier,
        color = containerColor,
        shape = RoundedCornerShape( size = 16.dp)
    ) {
        Column(
            modifier = Modifier.padding(vertical = 12.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text(
                text = count.toString(),
                style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.ExtraBold),
                color = contentColor
            )
            Text(
                text = label,
                style = MaterialTheme.typography.labelLarge,
                color = contentColor.copy(alpha = 0.8f)
            )
        }
    }
}
```

```
@Composable
fun TaskSummaryCard(todo: Int, completed: Int, total: Int) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 12.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        SummaryItem(
            label = "Active", count = todo, modifier = Modifier.weight(1f),
            containerColor = MaterialTheme.colorScheme.primaryContainer,
            contentColor = MaterialTheme.colorScheme.onPrimaryContainer
        )
        SummaryItem(
            label = "Done", count = completed, modifier = Modifier.weight(1f),
            containerColor = MaterialTheme.colorScheme.tertiaryContainer,
            contentColor = MaterialTheme.colorScheme.onTertiaryContainer
        )
        SummaryItem(
            label = "Total", count = total, modifier = Modifier.weight(1f),
            containerColor = MaterialTheme.colorScheme.secondaryContainer,
            contentColor = MaterialTheme.colorScheme.onSecondaryContainer
        )
    }
}
```

Langkah Pembuatan

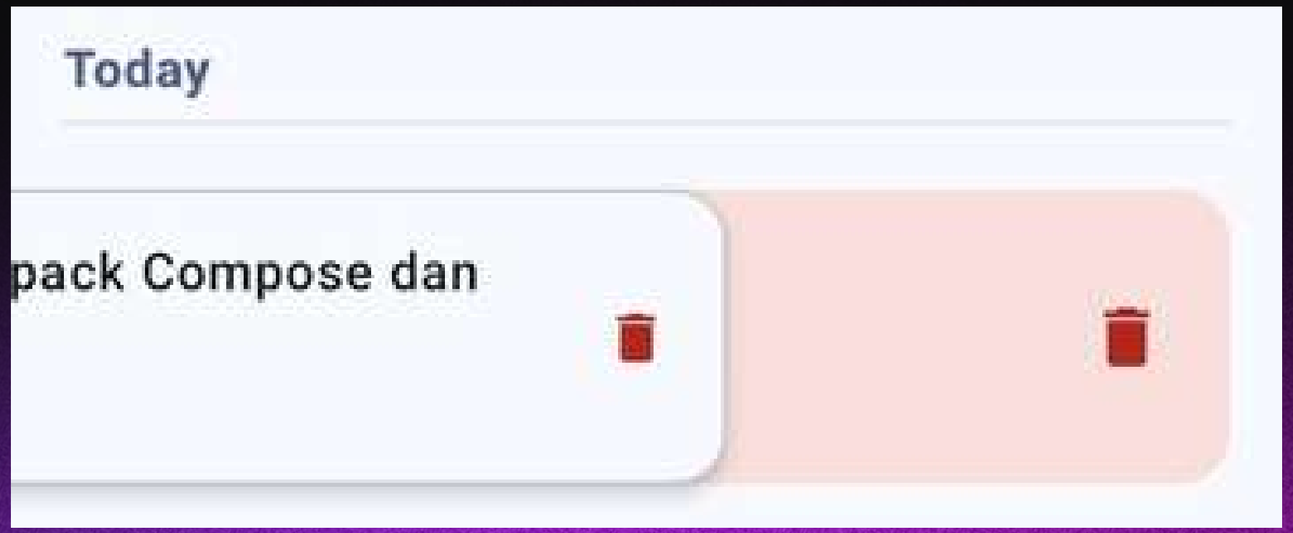
Buat TaskItem, komponen yang menampilkan task

```
Card(
    modifier = Modifier
        .fillMaxWidth()
        .clickable { onClick() },
    shape = RoundedCornerShape( size = 16.dp),
    colors = CardDefaults.cardColors(
        containerColor = if (task.isCompleted)
            MaterialTheme.colorScheme.surfaceContainerLow
        else
            MaterialTheme.colorScheme.surface
    ),
    elevation = CardDefaults.cardElevation(defaultElevation = if (task.isCompleted) 0.dp else 5.dp),
    border = if (isPast)
        BorderStroke( width = 2.dp,  color = MaterialTheme.colorScheme.error.copy(alpha = 0.8f))
    else
        BorderStroke( width = 1.dp,  color = MaterialTheme.colorScheme.outlineVariant)
) {
    Row(
        modifier = Modifier
            .padding( all = 16.dp)
            .fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {

        IconButton(
            onClick = { onCheckedChange(!task.isCompleted) },
            modifier = Modifier.size(24.dp)
        ) {
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .clip( shape = RoundedCornerShape( size = 6.dp))
                    .background(
                        color = if (task.isCompleted) MaterialTheme.colorScheme.primary
                        else MaterialTheme.colorScheme.outline.copy(alpha = 0.1f)
                    ),
                contentAlignment = Alignment.Center
            ) {
                if (task.isCompleted) {
                    Icon(
```

```
                    Icon(
                        imageVector = Icons.Default.Check,
                        contentDescription = null,
                        modifier = Modifier.size(16.dp),
                        tint = MaterialTheme.colorScheme.onPrimary
                    )
                }
            }
        }

        Column(modifier = Modifier.weight(1f).padding(horizontal = 16.dp)) {
            Text(
                text = task.title,
                style = MaterialTheme.typography.bodyLarge.copy(
                    fontWeight = FontWeight.Medium,
                    textDecoration = if (task.isCompleted) TextDecoration.LineThrough else null
                ),
                color = if (task.isCompleted)
                    MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.6f)
                else
                    MaterialTheme.colorScheme.onSurface
            )

            val deadlineText = if (showFullDate || task.isCompleted) {
                task.deadline.format( formatter = DateTimeFormatter.ofPattern("dd MMM, HH:mm"))
            } else {
                task.deadline.format( formatter = DateTimeFormatter.ofPattern("HH:mm"))
            }

            Text(
                text = deadlineText,
                style = MaterialTheme.typography.labelMedium,
                color = if (isPast) MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.onSurfaceVariant
            )
        }

        IconButton(onClick = onDelete, modifier = Modifier.size(24.dp)) {
            Icon(
```

Langkah Pembuatan

Buat fungsi *swipe to dismiss* pada task item

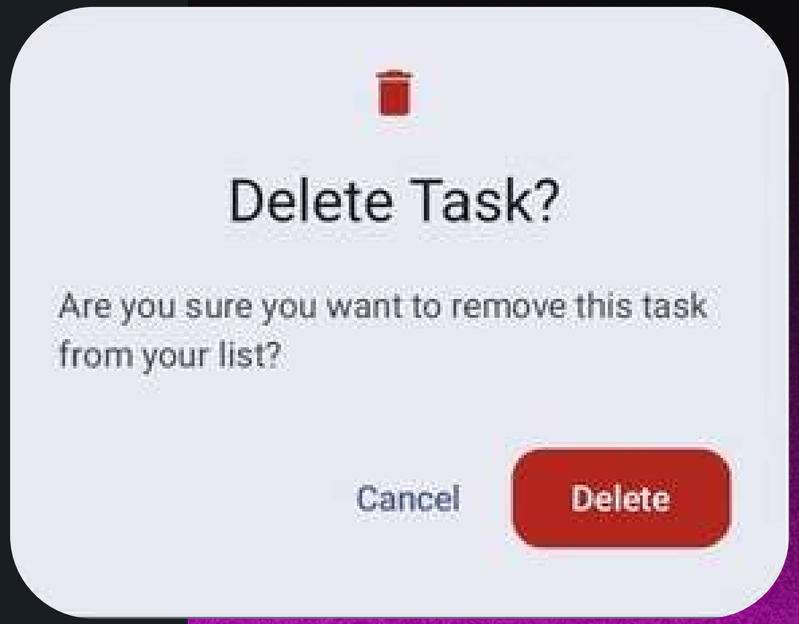



```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun TaskItem(
    task: Task,
    onCheckedChange: (Boolean) -> Unit,
    onDelete: () -> Unit,
    onClick: () -> Unit,
    showFullDate: Boolean = false
) {
    val isPast = task.deadline.isBefore( other = LocalDateTime.now()) && !task.isCompleted

    val dismissState = rememberSwipeToDismissBoxState(
        confirmValueChange = {
            if (it == SwipeToDismissBoxValue.EndToStart) {
                onDelete()
            }
            false
        }
    )

    SwipeToDismissBox(
        state = dismissState,
        enableDismissFromStartToEnd = false,
        backgroundContent = {
            val color = if (dismissState.dismissDirection == SwipeToDismissBoxValue.EndToStart) {
                MaterialTheme.colorScheme.errorContainer
            } else Color.Transparent

            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .clip( shape = RoundedCornerShape( size = 16.dp))
                    .background(color)
                    .padding(horizontal = 20.dp),
                contentAlignment = Alignment.CenterEnd
            ) {
                Icon(
                    imageVector = Icons.Default.Delete,
                    contentDescription = "Delete",
                    tint = MaterialTheme.colorScheme.error
                )
            }
```

Langkah Pembuatan

Buat DeleteConfirmDialog

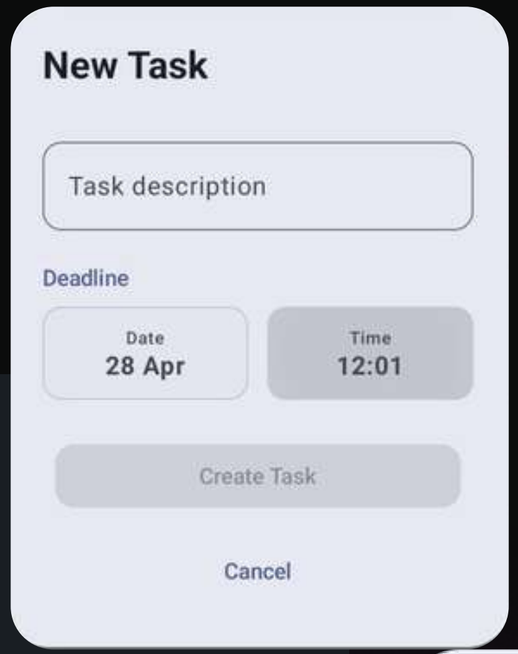
```
@Composable
fun DeleteConfirmDialog(onDismiss: () -> Unit, onConfirm: () -> Unit) {
    AlertDialog(
        onDismissRequest = onDismiss,
        icon = { Icon( imageVector = Icons.Default.Delete, contentDescription = null, tint = MaterialTheme.colorScheme.error) },
        title = { Text("Delete Task?") },
        text = { Text("Are you sure you want to remove this task from your list?") },
        confirmButton = {
            Button(
                onClick = onConfirm,
                colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error),
                shape = RoundedCornerShape( size = 12.dp)
            ) {
                Text("Delete")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        },
        shape = RoundedCornerShape( size = 28.dp)
    )
}
```

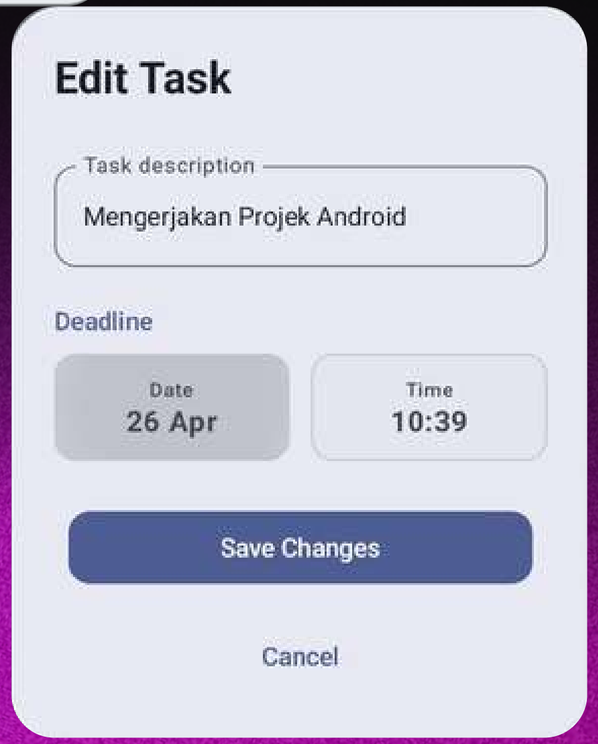


Langkah Pembuatan

Buat TaskDialog, pop-up ketika edit dan tambah task

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun TaskDialog(
    task: Task? = null,
    onDismiss: () -> Unit,
    onConfirm: (String, LocalDateTime) -> Unit
) {
    var title by remember { mutableStateOf( value = task?.title ?: "") }
    var selectedDate by remember { mutableStateOf( value = task?.deadline?.toLocalDate() ?: LocalDate.now()) }
    var selectedTime by remember { mutableStateOf( value = task?.deadline?.toLocalTime()?.withSecond(0)?.withNano( nanoOfSecond =

    var showDatePicker by remember { mutableStateOf( value = false) }
    var showTimePicker by remember { mutableStateOf( value = false) }

    val datePickerState = rememberDatePickerState(
        initialSelectedDateMillis = selectedDate.atStartOfDay(ZoneId.systemDefault()).toInstant().toEpochMilli()
    )
    val timePickerState = rememberTimePickerState(
        initialHour = selectedTime.hour,
        initialMinute = selectedTime.minute
    )

    AlertDialog(
        onDismissRequest = onDismiss,
```

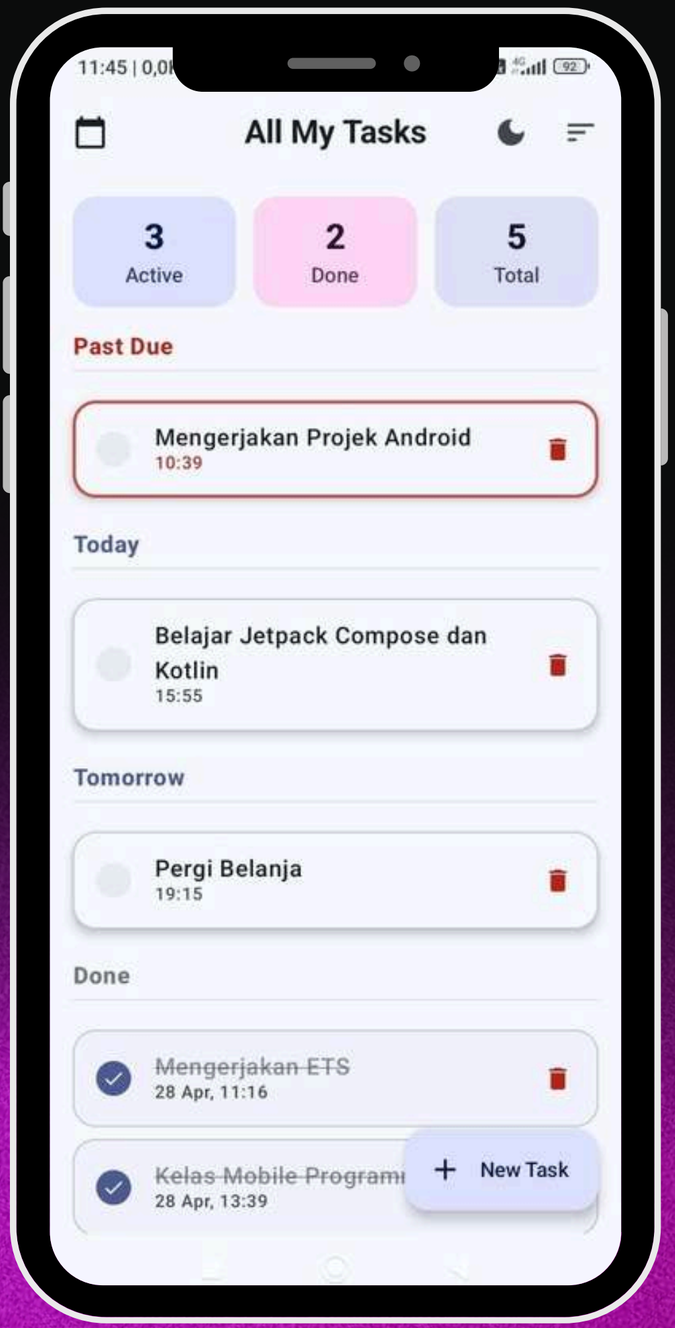




Langkah Pembuatan

Buat TaskScreen yang menggabungkan semuanya

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun TaskScreen(viewModel: TaskViewModel) {
    val tasks by viewModel.tasks
    val sortOrder by viewModel.sortOrder
    val viewMode by viewModel.viewMode
    val selectedDate by viewModel.selectedDate

    var showAddDialog by remember { mutableStateOf( value = false) }
    var taskToEdit by remember { mutableStateOf<Task?>( value = null) }
    var taskToDelete by remember { mutableStateOf<Task?>( value = null) }
    var showSortMenu by remember { mutableStateOf( value = false) }
    var showDatePickerForMode by remember { mutableStateOf( value = false) }

    val filteredTasks = if (viewMode == TaskViewModel.ViewMode.DAILY) {
        tasks.filter { it.deadline.toLocalDate() == selectedDate }
    } else {
        tasks
    }

    val pendingTasks = filteredTasks.filter { !it.isCompleted }
    val doneTasks = filteredTasks.filter { it.isCompleted }

    Scaffold(
        topBar = {
            Column {
                CenterAlignedTopAppBar(
                    title = {
                        Text(
                            if (viewMode == TaskViewModel.ViewMode.DAILY) "Daily Focus" else "All My Tasks",
                            style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
                        )
                    },
                    navigationIcon = {
                        IconButton(onClick = {
                            val nextMode = if (viewMode == TaskViewModel.ViewMode.ALL)
                                TaskViewModel.ViewMode.DAILY else TaskViewModel.ViewMode.ALL
```



Thank you for your attention